SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

LEARN SMART. WRITE BETTER QUERIES. ANALYZE DATA LIKE A PRO!

1. WHAT IS SQL?

SQL (Structured Query Language) is a standard language used to create, manage, and retrieve data from relational databases. It helps you communicate with databases.

2. DATABASE BASICS

Data is organized as:

DATABASE	TABLE	ROW	COLUMN
Holds related tables.	Collection of related data.	A single record (horizontal).	A field/attribute (vertical).

3. SQL STATEMENT STRUCTURE

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

← What to retrieve
← From which table
← Filter condition

Every SQL query usually starts with **SELECT** and follows this basic structure.

4. SQL DATA TYPES

TYPE	DESCRIPTION
INT	Whole numbers
DECIMAL(p,s)	Fixed-point numbers (e.g., 10.2)
VARCHAR(n)	Variable-length string (e.g., up to n chars)
CHAR(n)	Fixed-length string (e.g., n chars)
DATE	Date (YYYY-MM-DD)
DATETIME	Date and time
BOOLEAN	TRUE / FALSE

5. KEYS IN SQL

PRIMARY KEY
Uniquely identifies each record in a table. No NULL allowed.

FOREIGN KEY
References the PRIMARY KEY in another table. Maintains relationship between tables.

6. CONSTRAINTS

- **PRIMARY KEY** — Ensures unique & not null values.
- **FOREIGN KEY** — Maintains referential integrity.
- **NOT NULL** — Column cannot have NULL values.
- **UNIQUE** — All values in a column must be unique.
- **CHECK** — Ensures values satisfy a condition.
- **DEFAULT** — Sets a default value if none is provided.

7. SQL COMMAND CATEGORIES

DDL (Data Definition Language)
CREATE, ALTER, DROP, TRUNCATE

DML (Data Manipulation Language)
INSERT, UPDATE, DELETE

DQL (Data Query Language)
SELECT

DCL (Data Control Language)
GRANT, REVOKE

TCL (Transaction Control Language)
COMMIT, ROLLBACK, SAVEPOINT

QUICK TIP: Practice writing queries daily. The more you write, the better you get!

Created by VINAYAK OJHA

Keep Learning. Keep Growing!

★ ★ CONSISTENT PRACTICE = STRONG SQL SKILLS ★ ★ ★ | Stay Curious. Keep Exploring Data!

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

WRITE BETTER QUERIES. GET BETTER INSIGHTS!

2. SELECT & DISTINCT

2.1 SELECT

The **SELECT** statement is used to retrieve data from a table.

SYNTAX:

```
SELECT column1, column2, ...
FROM table_name;
```

EXAMPLE:

```
SELECT id, name, salary
FROM employees;
```

id	name	salary
1	Amit	55000
2	Neha	60000
3	Ravi	45000

Returns selected columns only.

2.2 SELECT *

Use * to select all columns from a table.

SYNTAX:

```
SELECT *
FROM table_name;
```

EXAMPLE:

```
SELECT *
FROM employees;
```

id	name	salary	dept	email
1	Amit	55000	HR	amit@example.com
2	Neha	60000	IT	neha@example.com
3	Ravi	45000	Finance	ravi@example.com

(All columns returned)

2.3 COLUMN ALIAS (AS)

Use **AS** to give a temporary name to a column or expression.

SYNTAX:

```
SELECT column_name AS alias_name
FROM table_name;
```

EXAMPLE:

```
SELECT name AS employee_name,
       salary AS monthly_salary
FROM employees;
```

employee_name	monthly_salary
Amit	55000
Neha	60000
Ravi	45000

Aliases make output easy to read and use.

2.4 DISTINCT

Use **DISTINCT** to return only unique (different) values.

SYNTAX:

```
SELECT DISTINCT column_name
FROM table_name;
```

EXAMPLE:

```
SELECT DISTINCT dept
FROM employees;
```

dept
HR
IT
Finance
Marketing

DISTINCT helps in removing duplicates from the result set.

2.5 CALCULATED COLUMNS

Perform calculations and return results as columns.

EXAMPLE:

```
SELECT name,
       salary,
       salary + 12 AS annual_salary,
       salary + 5000 AS bonus
FROM employees;
```

name	salary	annual_salary	bonus
Amit	55000	55012	60000
Neha	60000	60012	65000
Ravi	45000	45012	50000

2.6 BASIC ARITHMETIC OPERATIONS

OPERATOR	MEANING	EXAMPLE	RESULT
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 2	5
%	Modulo (Remainder)	10 % 3	1

2.7 COMMENTS

Comments are used to add notes in SQL queries.

```
--  Single line comment
SELECT * FROM employees; -- This gets all employees
/* Multi-line comment
    This is used for multiple lines */
SELECT * FROM employees;
```

QUICK TIPS

- Always list only the columns you need.
- Use aliases to make column names meaningful.
- Use DISTINCT to remove duplicates.
- Practice daily to write better queries!

DATA ANALYST USE

- Extract specific data for analysis.
- Clean and prepare data.
- Get unique values for filtering and segmentation.
- Make reports clear and accurate.

CONSISTENT PRACTICE = STRONG SQL SKILLS

VINAYAK OJHA

KEEP LEARNING. KEEP GROWING!

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

FILTER RIGHT. GET PRECISE RESULTS!

3. WHERE & FILTERING

1 WHERE

Filters records that meet a condition.

SYNTAX:

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

EXAMPLE:

```
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

name	salary
Neha	60000
Ravi	75000

2 AND

Returns records when all conditions are TRUE.

SYNTAX:

```
SELECT column1, column2
FROM table_name
WHERE condition1 AND
condition2;
```

EXAMPLE:

```
SELECT * FROM employees
WHERE salary > 50000
AND dept = 'IT';
```

id	name	salary	dept
2	Neha	60000	IT

3 OR

Returns records when any one condition is TRUE.

SYNTAX:

```
SELECT column1, column2
FROM table_name
WHERE condition1 OR
condition2;
```

EXAMPLE:

```
SELECT * FROM employees
WHERE dept = 'HR'
OR dept = 'Finance';
```

id	name	dept
1	Amit	HR
3	Ravi	Finance

4 NOT

Returns records when the condition is NOT TRUE.

SYNTAX:

```
SELECT column1, column2
FROM table_name
WHERE NOT condition;
```

EXAMPLE:

```
SELECT * FROM employees
WHERE NOT dept = 'IT';
```

id	name	dept
1	Amit	HR
3	Ravi	Finance

5 IN

Matches any value in a list of values.

SYNTAX:

```
SELECT column1, column2
FROM table_name
WHERE column_name IN
(value1, value2, ...);
```

EXAMPLE:

```
SELECT * FROM employees
WHERE dept IN ('HR', 'IT');
```

id	name	dept
1	Amit	HR
2	Neha	IT

6 NOT IN

Matches values that are NOT in a list.

SYNTAX:

```
SELECT column1, column2
FROM table_name
WHERE column_name NOT IN
(value1, value2, ...);
```

EXAMPLE:

```
SELECT * FROM employees
WHERE dept NOT IN ('HR', 'IT');
```

id	name	dept
3	Ravi	Finance

7 BETWEEN

Filters records within a range (inclusive).

SYNTAX:

```
SELECT column1, column2
FROM table_name
WHERE column_name
BETWEEN value1 AND value2;
```

EXAMPLE:

```
SELECT * FROM employees
WHERE salary BETWEEN 50000
AND 70000;
```

id	name	salary
1	Amit	55000
2	Neha	60000

8 LIKE

Searches for a pattern in a column.

SYNTAX:

```
SELECT column1, column2
FROM table_name
WHERE column_name LIKE
'pattern';
```

EXAMPLE:

```
SELECT * FROM employees
WHERE name LIKE 'A%';
```

id	name
1	Amit

9 WILDCARDS IN LIKE

WILDCARD	MEANING
%	Represents zero or more characters
_ (underscore)	Represents exactly one character

EXAMPLES:

PATTERN	MATCHES
'A%'	Amit, Ankit, Ajay
'%a'	Neha, Ravi, Pooja
'%it%'	Amit, Mohit, Ankit
'_avi'	Ravi, Kavi
'A_t'	Ait, Ant

10 NUMERIC COMPARISON OPERATORS

OPERATOR	MEANING	EXAMPLE
=	Equal to	salary = 50000
!= or <>	Not equal to	salary <> 50000
>	Greater than	salary > 50000
<	Less than	salary < 50000
>=	Greater than or equal to	salary >= 50000
<=	Less than or equal to	salary <= 50000

Use numeric operators for numbers, dates and other comparable values.

11 DATE FILTERING EXAMPLES

EXAMPLE 1: Between Dates

```
SELECT * FROM employees
WHERE join_date BETWEEN '2024-01-01'
                AND '2024-12-31';
```

EXAMPLE 2: Before a Date

```
SELECT * FROM employees
WHERE join_date < '2024-06-01';
```

EXAMPLE 3: After a Date

```
SELECT * FROM employees
WHERE join_date > '2024-01-01';
```

QUICK TIPS

- Use WHERE to filter early.
- Combine conditions with AND, OR, NOT.
- Use IN for multiple values.
- Use BETWEEN for ranges.
- Use LIKE with wildcards for text search.

DATA ANALYST USE

- Filter data for meaningful insights.
- Segment customers and sales.
- Clean and validate data.
- Find patterns and trends.
- Make reports accurate and precise.

Created by
VINAYAK OJHA

Keep Learning. Keep Growing!

LEARN PRACTICE SUCCEED

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

SORT SMART. LIMIT RIGHT. GET RESULTS FAST!

4. SORTING & LIMITING RESULTS

1 ORDER BY

Sorts the result set by one or more columns.

SYNTAX:

```
SELECT column1, column2
FROM table_name
ORDER BY column_name;
```

EXAMPLE:

```
SELECT name, salary
FROM employees
ORDER BY salary;
```

name	salary
Ravi	45000
Amit	55000
Neha	60000

↑ Ascending (Default)

2 ASC (Ascending)

Sorts in ascending order (smallest to largest / A to Z).

SYNTAX:

```
SELECT column1, column2
FROM table_name
ORDER BY column_name ASC;
```

EXAMPLE:

```
SELECT name, salary
FROM employees
ORDER BY salary ASC;
```

name	salary
Ravi	45000
Amit	55000
Neha	60000

↑ Smallest to Largest

3 DESC (Descending)

Sorts in descending order (largest to smallest / Z to A).

SYNTAX:

```
SELECT column1, column2
FROM table_name
ORDER BY column_name DESC;
```

EXAMPLE:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

name	salary
Neha	60000
Amit	55000
Ravi	45000

↓ Largest to Smallest

4 MULTIPLE COLUMN SORTING

Sort by multiple columns in a specific order.

SYNTAX:

```
SELECT column1, column2, column3
FROM table_name
ORDER BY column_name1 ASC, column_name2 DESC;
```

EXAMPLE:

```
SELECT name, dept, salary
FROM employees
ORDER BY depry ASC, salary DESC;
```

dept	name	salary
HR	Amit	55000
HR	Neha	50000
IT	Ravi	60000
IT	Sita	45000

★ First sorted by dept (A to Z). Then by salary (High to Low) within each dept.

5 LIMIT

Limits the number of rows returned.

SYNTAX:

```
SELECT column1, column2
FROM table_name
LIMIT number;
```

EXAMPLE:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

name	salary
Neha	60000
Amit	55000
Ravi	45000

★ Returns only top 3 rows.

6 TOP (SQL Server)

Returns the top N rows.

SYNTAX:

```
SELECT TOP number column1, column2
FROM table_name
ORDER BY column_name DESC;
```

EXAMPLE:

```
SELECT TOP 3 name, salary
FROM employees
ORDER BY salary DESC;
```

name	salary
Neha	60000
Amit	55000
Ravi	45000

★ Returns top N rows based on sorting.

7 OFFSET (Skip Rows)

Skips a number of rows.

SYNTAX:

```
SELECT column1, column2
FROM table_name
ORDER BY column_name
OFFSET number ROWS;
```

EXAMPLE:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
OFFSET 2 ROWS;
```

name	salary
Ravi	45000
Sita	40000
Arun	35000

★ Skips first 2 rows (after sorting) and returns the rest.

8 PAGINATION (LIMIT + OFFSET)

Used to fetch a specific page of results.

SYNTAX:

```
SELECT column1, column2
FROM table_name
ORDER BY column_name
LIMIT number OFFSET number;
```

EXAMPLE:

```
-- Page 2, 2 rows per page
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 2 OFFSET 2;
```

name	salary
Ravi	45000
Sita	40000

★ Page size = 2, Page number = 2 (Skips first 2 rows, returns next 2 rows)

9 ALIAS IN ORDER BY

You can use column alias in ORDER BY.

EXAMPLE:

```
SELECT name, salary AS emp_salary
FROM employees
ORDER BY emp_salary DESC;
```

name	emp_salary
Neha	60000
Amit	55000
Ravi	45000

★ Alias helps to make query more readable.

QUICK TIPS

- ORDER BY is used to sort data.
- ASC is default. DESC for reverse order.
- LIMIT / TOP is used to return limited rows.
- OFFSET is used to skip rows.
- Use LIMIT + OFFSET for pagination.

DATA ANALYST USE

- Sort sales, revenue, customers, etc.
- Get top N performing products.
- Implement leaderboards.
- Build pagination in dashboards.
- Improve report performance.

Created by
VINAYAK OJHA

Keep Learning. Keep Growing!

LEARN PRACTICE SUCCEED

★ ★ CONSISTENT PRACTICE = STRONG SQL SKILLS ★ ★ | Stay Curious. Keep Exploring Data!

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

AGGREGATE SMART. ANALYZE BETTER!

5. AGGREGATE FUNCTIONS, GROUP BY & HAVING

1 COUNT()

Counts the number of rows.

SYNTAX:

```
COUNT(column_name)
or
COUNT(*)
```

EXAMPLE:

```
SELECT COUNT(*)
AS total_employees
FROM employees;
```

total_employees
6

2 SUM()

Returns the total (sum) of values.

SYNTAX:

```
SUM(column_name)
```

EXAMPLE:

```
SELECT SUM(salary)
AS total_salary
FROM employees;
```

total_salary
338000

3 AVG()

Returns the average (mean) of values.

SYNTAX:

```
AVG(column_name)
```

EXAMPLE:

```
SELECT AVG(salary)
AS avg_salary
FROM employees;
```

avg_salary
56333.33

4 MIN()

Returns the minimum (lowest) value.

SYNTAX:

```
MIN(column_name)
```

EXAMPLE:

```
SELECT MIN(salary)
AS min_salary
FROM employees;
```

min_salary
40000

5 MAX()

Returns the maximum (highest) value.

SYNTAX:

```
MAX(column_name)
```

EXAMPLE:

```
SELECT MAX(salary)
AS max_salary
FROM employees;
```

max_salary
90000

SAMPLE TABLE: employees

id	name	dept	salary
1	Amit	HR	55000
2	Neha	IT	60000
3	Ravi	Finance	45000
4	Sita	HR	40000
5	Arjun	IT	72000
6	Pooja	Finance	90000

AGGREGATE FUNCTIONS perform calculations on a set of values and return a single result.

TIP: Use COUNT(*) to count all rows, including NULL values.

6. GROUP BY

Groups rows that have the same values in specified columns into summary rows.

SYNTAX:

```
SELECT column_name, AGG_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

EXAMPLE:

```
SELECT dept, COUNT(*) AS total_emp
FROM employees
GROUP BY dept;
```

dept	total_emp
HR	2
IT	2
Finance	2

GROUP BY is used with aggregate functions to summarize data.

7. HAVING

Filters **groups** created by **GROUP BY**. It is applied after aggregation.

SYNTAX:

```
SELECT column_name, AGG_FUNCTION(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

EXAMPLE:

```
SELECT dept, COUNT(*) AS total_emp
FROM employees
GROUP BY dept
HAVING COUNT(*) > 1;
```

dept	total_emp
HR	2
IT	2
Finance	2

HAVING is used to filter groups, not individual rows.

8. WHERE vs. HAVING

WHERE	BASIS	HAVING
Filters individual rows before grouping	Applied	Filters groups after grouping
Cannot be used with aggregate functions	Use With	Used with aggregate functions
Improves performance (by reducing data early)	Performance	Runs after GROUP BY, may be slower
Example: salary > 50000	Example	Example: COUNT(*) > 2

9. PRACTICAL EXAMPLES

1 Total salary by department

```
SELECT dept, SUM(salary) AS total_salary
FROM employees
GROUP BY dept;
```

dept	total_salary
HR	95000
IT	132000
Finance	135000

2 Departments having more than 1 employee

```
SELECT dept, COUNT(*) AS total_emp
FROM employees
GROUP BY dept
HAVING COUNT(*) > 1;
```

dept	total_emp
HR	2
IT	2
Finance	2

QUICK TIPS

- Always use aliases for better readability.
- Aggregate functions ignore NULL values (except COUNT(*)).
- Use GROUP BY with aggregate functions.
- Use HAVING to filter groups, not rows.

DATA ANALYST USE

- Total sales by category
- Average order value
- Customer count by region
- Monthly revenue summary
- Performance reporting
- Trend and summary analysis

Created by
VINAYAK OJHA

Keep Learning. Keep Growing!

LEARN PRACTICE SUCCEED

CONSISTENT PRACTICE = STRONG SQL SKILLS | Stay Curious. Keep Exploring Data!

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

CONNECT DATA. REVEAL INSIGHTS!

6. SQL JOINS

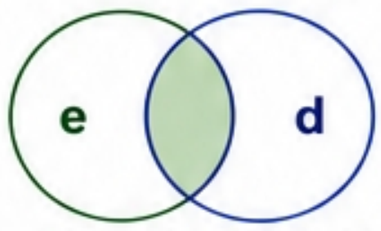
JOINs are used to combine rows from two or more tables based on a related column between them.

Table: employees (e)

emp_id	name	dept_id
1	Amit	10
2	Neha	20
3	Ravi	10
4	Sita	30

Table: departments (d)

dept_id	dept_name
10	HR
20	IT
30	Finance
40	Marketing

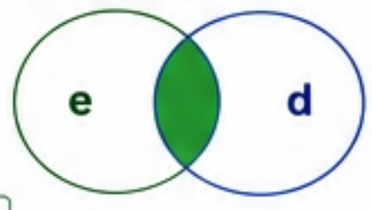


e.dept_id = d.dept_id

e = employees
d = departments
Overlap = Matched Rows

1 INNER JOIN

Returns only matching rows from both tables.



SYNTAX:

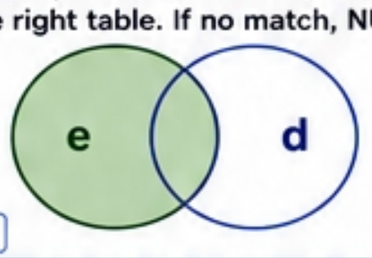
```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

EXAMPLE RESULT:

emp_id	name	dept_name
1	Amit	HR
2	Neha	IT
3	Ravi	HR
4	Sita	Finance

2 LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table (employees) and matching rows from the right table. If no match, NULL.



SYNTAX:

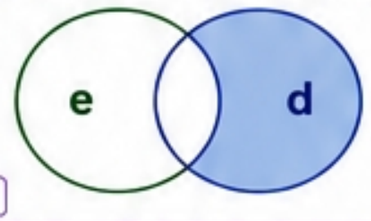
```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

EXAMPLE RESULT:

emp_id	name	dept_name
1	Amit	HR
2	Neha	IT
3	Ravi	HR
4	Sita	Finance

3 RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table (departments) and matching rows from the left table. If no match, NULL.



SYNTAX:

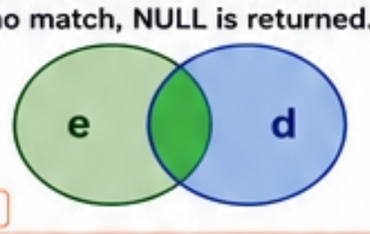
```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

EXAMPLE RESULT:

emp_id	name	dept_name
1	Amit	HR
2	Neha	IT
3	Ravi	HR
4	Sita	Finance
NULL	NULL	Marketing

4 FULL OUTER JOIN

Returns all rows from both tables. If no match, NULL is returned.



SYNTAX:

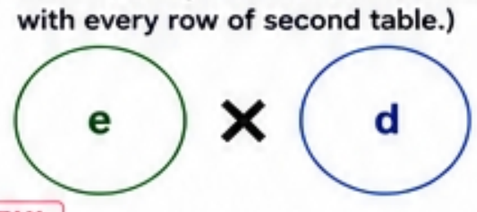
```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d
ON e.dept_id = d.dept_id;
```

EXAMPLE RESULT:

emp_id	name	dept_name
1	Amit	HR
2	Neha	IT
3	Ravi	HR
4	Sita	Finance
NULL	NULL	Marketing

5 CROSS JOIN

Returns the Cartesian product of both tables. (Every row of first table matches with every row of second table.)



SYNTAX:

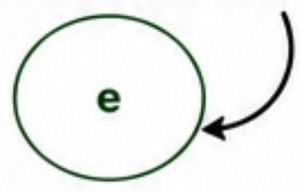
```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

EXAMPLE RESULT:

emp_id	name	dept_name
1	Amit	HR
1	Amit	IT
1	Amit	Finance
1	Amit	Marketing
... many more rows		

6 SELF JOIN

A table is joined with itself to compare rows within the same table.



SYNTAX:

```
SELECT e1.name AS emp, e2.name AS manager
FROM employees e1
JOIN employees e2
ON e1.manager_id = e2.emp_id;
```

EXAMPLE RESULT:

emp	manager
Neha	Amit
Ravi	Amit
Sita	Neha

(Based on manager_id relationship in the same table)

JOIN SYNTAX QUICK REFERENCE

JOIN TYPE	
INNER JOIN	`SELECT ... FROM A INNER JOIN B ON A.key = B.key;`
LEFT JOIN	`SELECT ... FROM A LEFT JOIN B ON A.key = B.key;`
RIGHT JOIN	`SELECT ... FROM A RIGHT JOIN B ON A.key = B.key;`
FULL OUTER JOIN	`SELECT ... FROM A FULL OUTER JOIN B ON A.key = B.key;`
CROSS JOIN	`SELECT ... FROM A CROSS JOIN B;`
SELF JOIN	`SELECT ... FROM A a1 JOIN A a2 ON A1. Bkey = a2.key;`

WHEN TO USE WHICH JOIN?

INNER JOIN	When you want only matching records.
LEFT JOIN	When you want all records from the left table.
RIGHT JOIN	When you want all records from the right table.
FULL OUTER JOIN	When you want all records from both tables.
CROSS JOIN	When you need all possible combinations.
SELF JOIN	When you need to compare rows within the same table.

QUICK TIPS

- Always use table aliases (e, d) for readability.
- Use INNER JOIN when possible (faster).
- Avoid SELECT * in production queries.
- Always check join condition (ON clause).
- Indexes on join columns improve performance.

DATA ANALYST USE CASES

- Sales with customer details (INNER JOIN)
- All customers including non-buyers (LEFT JOIN)
- All products including non-sold (RIGHT JOIN)
- Data comparison & reconciliation (FULL JOIN)
- Combinations for analysis (CROSS JOIN)

Created by
VINAYAK OJHA

Keep Learning. Keep Growing!

LEARN PRACTICE SUCCEED

★ ★ CONSISTENT PRACTICE = STRONG SQL SKILLS ★ ★ | Stay Curious. Keep Exploring Data!

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

WRITE LESS. DO MORE. GET POWERFUL RESULTS!

7. ADVANCED SQL FUNCTIONS

1 STRING FUNCTIONS

FUNCTION	DESCRIPTION
UPPER(str)	Converts to uppercase
LOWER(str)	Converts to lowercase
LENGTH(str)	Returns length of string
SUBSTRING(str, start, length)	Extracts substring
TRIM(str)	Removes leading and trailing spaces
CONCAT(str1, str2)	Joins two strings

EXAMPLE:

```
SELECT UPPER(name), LENGTH(name)
FROM employees;
```

2 NUMERIC FUNCTIONS

FUNCTION	DESCRIPTION
ROUND(num, n)	Rounds to n decimal places
CEIL(num)	Rounds up to nearest integer
FLOOR(num)	Rounds down to nearest integer
ABS(num)	Returns absolute value
MOD(a, b)	Returns remainder of a / b

EXAMPLE:

```
SELECT salary, ROUND(salary, 2),
ABS(-salary), MOD(salary, 1000)
FROM employees;
```

3 DATE FUNCTIONS

FUNCTION	DESCRIPTION
NOW()	Current date and time
CURDATE()	Current date
YEAR(date)	Extract year
MONTH(date)	Extract month
DAY(date)	Extract day
DATEDIFF(d1, d2)	Days between two dates

EXAMPLE:

```
SELECT name, CURDATE() as today,
DATEDIFF(CURDATE(), join_date) as
days_worked
FROM employees;
```

4 CONDITIONAL FUNCTIONS

FUNCTION	DESCRIPTION
IF(condition, a, b)	Returns a if true, else b
CASE WHEN	Multiple conditions
COALESCE(a, b, ...)	Returns first non-NULL value
NULLIF(a, b)	Returns NULL if a = b

EXAMPLE:

```
SELECT name, salary,
IF(salary > 50000, 'High', 'Low')
as salary_level
FROM employees;
```

5 WINDOW (ANALYTIC) FUNCTIONS

Used to perform calculations across a set of table rows related to the current row without collapsing rows like GROUP BY.

FUNCTION	DESCRIPTION
ROW_NUMBER()	Assigns unique sequential integer to rows
RANK()	Ranks rows with gaps for ties
DENSE_RANK()	Ranks rows without gaps for ties
LAG(col, n)	Access data from n rows before current row
LEAD(col, n)	Access data from n rows after current row
SUM(col) OVER(...)	Running total
AVG(col) OVER(...)	Running average

SYNTAX:

```
FUNCTION() OVER (
  PARTITION BY column_name
  ORDER BY column_name
  [ROWS BETWEEN ... AND ...]
)
```

EXAMPLE:

```
SELECT name, dept, salary,
ROW_NUMBER() OVER (PARTITION BY
dept ORDER BY salary DESC) as rn,
SUM(salary) OVER (PARTITION BY
dept) as dept_total
FROM employees;
```

EXAMPLE RESULT:

name	dept	salary	rn	dept_total
Amit	HR	55000	1	115000
Neha	HR	60000	2	115000
Ravi	IT	70000	1	145000
Sita	IT	75000	2	145000
Arjun	Finance	65000	1	155000
Pooja	Finance	90000	2	155000

★ Window functions are evaluated after FROM, WHERE, GROUP BY, HAVING but before ORDER BY.

6 SUBQUERIES

A query inside another query. Subqueries can be used in SELECT, FROM, WHERE, HAVING.

1 SUBQUERY IN WHERE

SYNTAX:

```
SELECT column1
FROM table1
WHERE column2 = (SELECT column
                 FROM table2
                 WHERE condition);
```

EXAMPLE:

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary)
                FROM employees);
```

2 SUBQUERY IN FROM

SYNTAX:

```
SELECT t.column1, t.column2
FROM (SELECT column1, column2
      FROM table1
      WHERE condition) t;
```

EXAMPLE:

```
SELECT t.dept, t.avg_salary
FROM (SELECT dept, AVG(salary) as
      avg_salary
      FROM employees
      GROUP BY dept) t;
```

3 CORRELATED SUBQUERY

SYNTAX:

```
SELECT column1
FROM table1 t1
WHERE column2 > (SELECT AVG(column2)
                 FROM table2 t2
                 WHERE t1.id = t2.id);
```

EXAMPLE:

```
SELECT name, salary
FROM employees e1
WHERE salary > (SELECT AVG(salary)
                FROM employees e2
                WHERE e1.dept = e2.dept);
```

4 EXISTS / NOT EXISTS

SYNTAX:

```
SELECT column1
FROM table1
WHERE EXISTS (SELECT 1
              FROM table2
              WHERE table2.id = table1.id);
```

EXAMPLE:

```
SELECT name
FROM employees e
WHERE EXISTS (SELECT 1
              FROM departments d
              WHERE d.dept_id = e.dept_id);
```

QUICK TIPS

- Use functions to make queries smarter.
- Window functions help in advanced analysis without using subqueries.
- Subqueries are powerful – use them wisely for filtering and comparisons.
- Always test your queries with small datasets first.

DATA ANALYST USE CASES

- String functions – data cleaning & formatting
- Numeric functions – calculations & metrics
- Date functions – trend analysis & reporting
- Window functions – rankings, running totals, moving averages
- Subqueries – filtering, comparisons, existence checks

Created by
VINAYAK OJHA

Keep Learning. Keep Growing!

LEARN PRACTICE SUCCEED

★ CONSISTENT PRACTICE = STRONG SQL SKILLS | Stay Curious. Keep Exploring Data!

SQL CHEAT SHEET

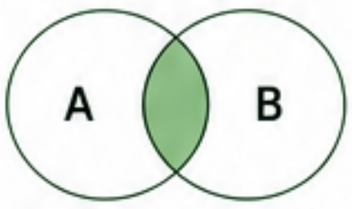
QUERIES • FUNCTIONS • JOINS • ANALYTICS

THINK LIKE A DATA ANALYST. SOLVE LIKE SQL!

8. SET OPERATORS & NULLS

1 UNION

Combines result sets from two queries and removes duplicates.



SYNTAX:

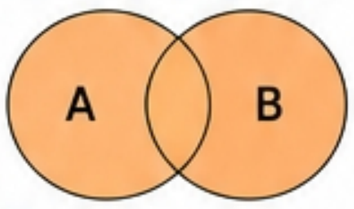
```
SELECT column_name(s) FROM tableA
UNION
SELECT column_name(s) FROM tableB;
```

EXAMPLE:

```
SELECT name FROM employees
UNION
SELECT name FROM contractors;
```

2 UNION ALL

Combines result sets from two queries and keeps duplicates.



SYNTAX:

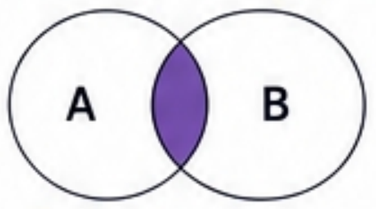
```
SELECT column_name(s) FROM tableA
UNION ALL
SELECT column_name(s) FROM tableB;
```

EXAMPLE:

```
SELECT name FROM employees
UNION ALL
SELECT name FROM contractors;
```

3 INTERSECT

Returns only rows that are common in both result sets.



SYNTAX:

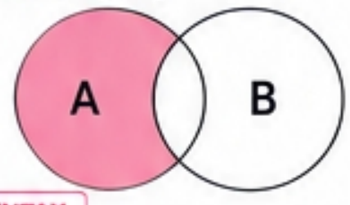
```
SELECT column_name(s) FROM tableA
INTERSECT
SELECT column_name(s) FROM tableB;
```

EXAMPLE:

```
SELECT name FROM employees
INTERSECT
SELECT name FROM contractors;
```

4 EXCEPT (MINUS)

Returns rows from the first query that are not in the second query.



SYNTAX:

```
SELECT column_name(s) FROM tableA
EXCEPT
SELECT column_name(s) FROM tableB;
```

EXAMPLE:

```
SELECT name FROM employees
EXCEPT
SELECT name FROM contractors;
```

5 NULL VALUES

NULL means "no value" or "unknown value".
NULL is not equal to anything, not even NULL.

COMPARISON WITH NULL

EXPRESSION	RESULT
column = NULL	Unknown (False)
column <> NULL	Unknown (False)
column IS NULL	True (if value is NULL)
column IS NOT NULL	True (if value is not NULL)

CHECKING NULL VALUES

IS NULL

Find rows where a column has NULL value.

SYNTAX:

```
SELECT column_name(s)
FROM table_name
WHERE column_name IS NULL;
```

EXAMPLE:

```
SELECT name, salary
FROM employees
WHERE salary IS NULL;
```

IS NOT NULL

Find rows where a column has a value (not NULL).

SYNTAX:

```
SELECT column_name(s)
FROM table_name
WHERE column_name IS NOT NULL;
```

EXAMPLE:

```
SELECT name, salary
FROM employees
WHERE salary IS NOT NULL;
```

TIPS ABOUT NULL

- Use IS NULL and IS NOT NULL to check for NULLs.
- NULL in arithmetic operations returns NULL.
- Use COALESCE() or IFNULL() to replace NULL with a value.

EXAMPLE:

```
SELECT COALESCE(salary, 0) AS
salary_after_replace
FROM employees;
```

9. COMMON SQL CLAUSES ORDER

The logical order in which SQL clauses are executed:

1 FROM - Get data from tables
2 WHERE - Filter rows
3 GROUP BY - Group rows
4 HAVING - Filter groups
5 SELECT - Select columns
6 ORDER BY - Sort result
7 LIMIT/OFFSET - Limit rows



★ Note: The order of writing in a query is different, but the execution order is as above.

10. USEFUL TIPS

- Always use meaningful aliases.
- Use WHERE to filter rows before grouping.
- Use HAVING to filter after grouping.
- Avoid SELECT * in production queries.
- Use LIMIT to improve performance.
- Test your queries with a small dataset.

11. SHORTCUT KEYWORDS

AS - Alias for column/table
DISTINCT - Remove duplicate rows
BETWEEN - Between a range
IN - Match any value in a list
LIKE - Pattern matching
TOP/LIMIT - Limit number of rows

12. SAMPLE PRACTICE QUERY

Find department-wise average salary of employees having salary > 50000.

```
SELECT dept, AVG(salary) AS avg_salary
FROM employees
WHERE salary > 50000
GROUP BY dept
HAVING AVG(salary) > 50000
ORDER BY avg_salary DESC;
```

KEEP PRACTICING!

Every query you write today makes you a better Data Analyst tomorrow.

Created by
VINAYAK OJHA
Keep Learning. Keep Growing!

LET'S CONNECT
linkedin.com/in/vinayak-ojha-845854322
vinayakojha2002@gmail.com

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

PRACTICE SMART. WRITE BETTER. BECOME A DATA ANALYST!

10. GROUP BY, HAVING & ORDER BY DEEP DIVE

1 GROUP BY

Groups rows that have the same values in specified columns.

SYNTAX:

```
SELECT column_name, AGG_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

EXAMPLE:

```
SELECT dept, COUNT(*) AS total_emp
FROM employees
GROUP BY dept;
```

RESULT:

dept	total_emp
HR	2
IT	2
Finance	2

GROUP BY is used with aggregate functions to summarize data.

2 HAVING

Filters groups created by GROUP BY. It is applied after aggregation.

SYNTAX:

```
SELECT column_name, AGG_FUNCTION(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

EXAMPLE:

```
SELECT dept, COUNT(*) AS total_emp
FROM employees
GROUP BY dept
HAVING COUNT(*) > 1;
```

RESULT:

dept	total_emp
HR	2
IT	2
Finance	2

HAVING is used to filter groups, not individual rows.

3 ORDER BY

Sorts the result set in ascending (ASC) or descending (DESC) order.

SYNTAX:

```
SELECT column1, columm2
FROM table_name
ORDER BY column1 ASC|DESC;
```

EXAMPLE:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

RESULT:

name	salary
Pooja	90000
Sita	75000
Ravi	70000
...	

Default order is ASC (ascending).

4 COMBINING GROUP BY, HAVING & ORDER BY

EXAMPLE QUERY:

```
SELECT dept, AVG(salary) AS avg_salary,
       COUNT(*) AS total_emp
FROM employees
WHERE salary > 40000
GROUP BY dept
HAVING AVG(salary) > 60000
ORDER BY avg_salary DESC;
```

EXECUTION ORDER:

1	FROM
2	WHERE (filters rows)
3	GROUP BY (groups rows)
4	HAVING (filters groups)
5	SELECT (selects columns)
6	ORDER BY (sorts result)

SAMPLE RESULT:

dept	avg_salary	total_emp
Finance	72500.00	2
IT	65000.00	2

★ Always remember: WHERE filters rows before grouping, HAVING filters groups after grouping.

5. ALIASES (AS)

Aliases are used to give a column or table a temporary name.

EXAMPLE:

```
SELECT name AS employee_name,
       salary AS employee_salary
FROM employees;
```

Aliases make queries more readable.

6. DISTINCT

Returns only distinct (unique) values.

EXAMPLE:

```
SELECT DISTINCT dept
FROM employees;
```

Useful to get unique values from a column.

7. LIMIT & OFFSET

Used to limit the number of rows returned.

SYNTAX:

```
SELECT column1, column2
FROM table_name
LIMIT number OFFSET number;
```

EXAMPLE:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3 OFFSET 1;
```

MEANING:

- LIMIT 3 – returns only 3 rows.
- OFFSET 1 – skips the first row.

8. CASE STATEMENT

Used for conditional logic in queries.

SYNTAX:

```
SELECT column_name,
CASE WHEN condition1 THEN result1
     WHEN condition2 THEN result2
     ELSE resultN
END AS new_column
FROM table_name;
```

EXAMPLE:

```
SELECT name, salary,
CASE
  WHEN salary > 70000 THEN 'High'
  WHEN salary > 50000 THEN 'Medium'
  ELSE 'Low'
END AS salary_level
FROM employees;
```

QUICK TIPS

- Use meaningful aliases.
- Apply WHERE before GROUP BY.
- Use HAVING after GROUP BY.
- Always ORDER BY for better readability.
- Use DISTINCT to remove duplicates.
- Test queries step by step.

COMMON USE CASES

- Sales analysis by region (GROUP BY)
- High salary employees (WHERE)
- Departments with avg salary > 60000 (HAVING)
- Top 5 highest paid employees (ORDER BY + LIMIT)
- Unique department list (DISTINCT)
- Conditional salary categories (CASE)

PRACTICE CHALLENGE

Write a query to find department-wise average salary of employees having salary > 50000 and show only those departments where average salary is greater than 60000. Sort the result by average salary in descending order.

Created by
VINAYAK OJHA
Keep Learning. Keep Growing!

CONNECT WITH ME
linkedin.com/in/vinayak-ojha-845854322
vinayakojha2002@gmail.com

★ REMEMBER
Good SQL skills = Better insights = Smarter decisions = Career growth!

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

MASTER SQL. ANALYZE DATA. DRIVE DECISIONS!

11. SQL PATTERNS, WILDCARDS & REGEXP

1 LIKE OPERATOR

Used in WHERE clause to search for a pattern.

WILDCARD	MEANING
%	Zero or more characters
_	Exactly one character

EXAMPLE:

```
SELECT name
FROM employees
WHERE name LIKE 'A%';    -- Starts with A
WHERE name LIKE '%a';    -- Ends with a
WHERE name LIKE '%an%'; -- Contains 'an'
WHERE name LIKE 'A_n%'; -- A + any 1 char + any
```

2 REGEXP (REGULAR EXPRESSION)

Used to match complex patterns.

PATTERN	DESCRIPTION
^	Starts with
$	Ends with
.	Any single character
[abc]	Any one character a, b or c
[a-z]	Any character in a to z
[^abc]	Any character except a, b or c
*	Zero or more occurrences
+	One or more occurrences
{n}	Exactly n occurrences
{n,}	n or more occurrences
{n,m}	Between n and m occurrences

EXAMPLE:

```
SELECT name FROM employees
WHERE name REGEXP '^A';     -- Starts with A
WHERE name REGEXP 'n$';     -- Ends with n
WHERE name REGEXP 'a.{2}i';-- a + any 2 chars + i
WHERE name REGEXP '^[A-Za-z]+$'; -- Only alphabets
```

3 COMMON PATTERN EXAMPLES

PATTERN	MATCHES
'^A'	Starts with A
'a$'	Ends with a
'^A.*a$'	Starts with A and ends with a
'.*man.*'	Contains 'man' anywhere
'^[0-9]+$'	Only numbers
'^[A-Za-z]+$'	Only letters
'^[A-Za-z0-9]+$'	Letters and numbers only
'^[A-Z][a-z]+$'	Starts with capital, then small letters

LIKE is simple and fast.
REGEXP is powerful and flexible.
Choose based on your need!

12. SUBQUERIES WITH IN / ANY / ALL

1 IN OPERATOR

Checks if a value matches any value in a subquery result.

EXAMPLE:

```
SELECT name, dept
FROM employees
WHERE dept IN (SELECT dept
               FROM departments);
```

IN is used when subquery returns multiple values.

2 ANY OPERATOR

Compares a value with any (one or more) values returned by subquery.

EXAMPLE:

```
SELECT name, salary
FROM employees
WHERE salary > ANY
      (SELECT salary
      FROM employees
      WHERE dept = 'IT');
```

> ANY means greater than at least one value.

3 ALL OPERATOR

Compares a value with all values returned by subquery.

EXAMPLE:

```
SELECT name, salary
FROM employees
WHERE salary > ALL
      (SELECT salary
      FROM employees
      WHERE dept = 'IT');
```

> ALL means greater than every value.

4 NOT IN OPERATOR

Selects values not present in subquery result.

EXAMPLE:

```
SELECT name, dept
FROM employees
WHERE dept NOT IN (SELECT dept
                   FROM departments
                   WHERE dept = 'HR');
```

NOT IN is used to exclude values from subquery.

13. AGGREGATE FUNCTIONS

Aggregate functions perform calculations on a set of values and return a single result.

FUNCTION	DESCRIPTION
COUNT()	Counts number of rows
SUM()	Returns total sum
AVG()	Returns average
MIN()	Returns minimum value
MAX()	Returns maximum value

EXAMPLE:

```
SELECT dept,
  COUNT(*) AS total_emp,
  SUM(salary) AS total_salary,
  AVG(salary) AS avg_salary,
  MIN(salary) AS min_salary,
  MAX(salary) AS max_salary
FROM employees
GROUP BY dept;
```

Aggregate functions are often used with GROUP BY to summarize data.

14. DATE & TIME FUNCTIONS

Built-in functions to work with dates and time.

FUNCTION	DESCRIPTION
NOW()	Current date and time
CURDATE()	Current date
CURTIME()	Current time
DATE()	Extracts date from datetime
YEAR(date)	Extracts year
MONTH(date)	Extracts month
DAY(date)	Extracts day
DATE_ADD(date, INTERVAL n DAY)	Adds n days to date
DATEDIFF(date1, date2)	Returns difference in days

EXAMPLE:

```
SELECT name,
  JOIN_DATE,
  YEAR(JOIN_DATE) AS join_year,
  MONTH(JOIN_DATE) AS join_month,
  DAY(JOIN_DATE) AS join_day,
  DATEDIFF(CURDATE(),
      JOIN_DATE) AS days_worked
FROM employees;
```

Always store dates in proper format (YYYY-MM-DD) for best results.

QUICK TIPS

- Use LIKE for simple pattern matching.
- Use REGEXP for complex pattern matching.
- Use IN for multiple possible values.
- Use ANY / ALL for advanced comparisons.
- Use NOT IN to exclude values.
- Use aggregate functions with GROUP BY.
- Use date functions for time-based analysis.
- Practice patterns regularly to get faster!

COMMON USE CASES

- Find departments with more than 5 employees.
- List employees whose salary is greater than average salary.
- Find employees who joined this month.
- Find customers who made purchases (using IN subquery).
- Find products not sold in a specific category (using NOT IN subquery).

BEST PRACTICES

- Write readable and well-indented SQL.
- Use aliases (AS) for better readability.
- Filter data early (WHERE before GROUP BY).
- Avoid SELECT * in production queries.
- Test your queries with small datasets first.
- Comment your complex queries.

Created by
VINAYAK OJHA
Keep Learning. Keep Growing!

CONNECT WITH ME
linkedin.com/in/vinayak-ojha-845854322
vinayakojha2002@gmail.com

REMEMBER
Good SQL skills = Better Insights
Better Insights = Better Decisions
Better Decisions = Career Growth!

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

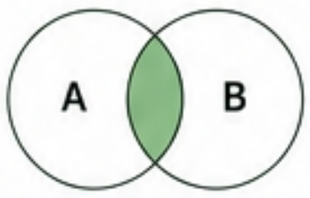
PRACTICE EVERY DAY. IMPROVE EVERY DAY. EXCEL IN SQL!

15. JOINS IN SQL

Joins are used to combine rows from two or more tables based on a related column.

1 INNER JOIN

Returns only matching rows from both tables.

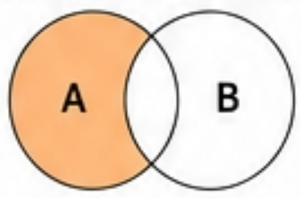


EXAMPLE:

```
SELECT A.name, B.salary
FROM employees A
INNER JOIN salaries B
ON A.id = B.emp_id;
```

2 LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from left table and matching rows from right table.

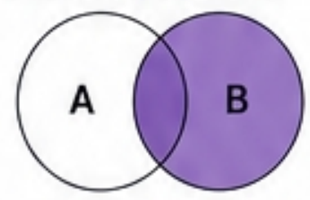


EXAMPLE:

```
SELECT A.name, B.salary
FROM employees A
LEFT JOIN salaries B
ON A.id = B.emp_id;
```

3 RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from right table and matching rows from left table.

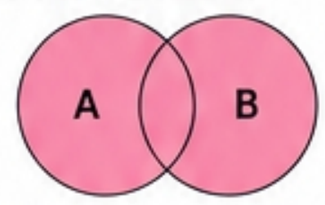


EXAMPLE:

```
SELECT A.name, B.salary
FROM employees A
RIGHT JOIN salaries B
ON A.id = B.emp_id;
```

4 FULL OUTER JOIN (FULL JOIN)

Returns all rows when there is a match in either left or right table.



EXAMPLE:

```
SELECT A.name, B.salary
FROM employees A
FULL OUTER JOIN salaries B
ON A.id = B.emp_id;
```

5 CROSS JOIN

Returns the Cartesian product of both tables (all possible combinations).

A x B	A x B	A x B
A x B	A x B	A x B
A x B	A x B	A x B

EXAMPLE:

```
SELECT A.name, B.salary
FROM employees A
CROSS JOIN salaries B;
```

★ TIP: Always use meaningful JOINs to get accurate and relevant results!

16. UNION & UNION ALL

UNION

Combines result sets and removes duplicates.

EXAMPLE:

```
SELECT name FROM employees
UNION
SELECT name FROM contractors;
```

UNION ALL

Combines result sets and keeps duplicates.

EXAMPLE:

```
SELECT name FROM employees
UNION ALL
SELECT name FROM contractors;
```

Use UNION when you need unique records.
Use UNION ALL when duplicates are allowed.

17. CTE (COMMON TABLE EXPRESSION)

Defines a temporary result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement.

EXAMPLE:

```
WITH high_salary AS (
    SELECT name, salary
    FROM employees
    WHERE salary > 50000
)
SELECT name, salary
FROM high_salary
ORDER BY salary DESC;
```

CTEs improve readability and help break complex queries into smaller parts.

18. VIEW

A virtual table based on the result-set of an SQL query.

CREATE VIEW

EXAMPLE:

```
CREATE VIEW emp_view AS
SELECT id, name, dept, salary
FROM employees;
```

USE VIEW

EXAMPLE:

```
SELECT * FROM emp_view
WHERE salary > 50000;
```

Views simplify complex queries and improve security by restricting data access.

QUICK SYNTAX RECAP

OPERATION	SYNTAX
SELECT	SELECT column1, column2 FROM table;
WHERE	SELECT * FROM table WHERE condition;
GROUP BY	SELECT col, COUNT(*) FROM table GROUP BY col;
HAVING	SELECT col, COUNT(*) FROM table GROUP BY col HAVING COUNT(*) > 1;
ORDER BY	SELECT * FROM table ORDER BY col ASC;
LIMIT	SELECT * FROM table LIMIT 10;
LIKE	SELECT * FROM table WHERE col LIKE 'A%';
IN	SELECT * FROM table WHERE col IN (1,2,3);
BETWEEN	SELECT * FROM table WHERE col BETWEEN 10 AND 20;

TOP SQL COMMANDS

- SELECT – Retrieve data
- FROM – Specify table
- WHERE – Filter data
- GROUP BY – Group rows
- HAVING – Filter groups
- ORDER BY – Sort result
- JOIN – Combine tables
- UNION – Combine results
- INSERT INTO – Add new data
- UPDATE – Modify data
- DELETE – Remove data
- CREATE TABLE – Create table
- DROP TABLE – Delete table

COMMON DATA TYPES

DATA TYPE	DESCRIPTION
INT	Whole numbers
BIGINT	Large whole numbers
VARCHAR(n)	Variable length string
CHAR(n)	Fixed length string
DATE	Date value (YYYY-MM-DD)
DATETIME	Date and time
DECIMAL(p,s)	Exact numeric (precision, scale)
FLOAT / REAL	Approximate numeric
BOOLEAN	True / False values
TEXT	Large text data

DAILY PRACTICE PLAN

- Practice basic SELECT queries
- Solve 5–10 problems daily
- Learn one new concept every day
- Build real-world projects
- Review and revise regularly

Created by
VINAYAK OJHA

Keep Learning. Keep Growing!

LEARN PRACTICE SUCCEED

CONNECT WITH ME
linkedin.com/in/vinayak-ojha-845854322
vinayakojha2002@gmail.com

REMEMBER
Good SQL skills = Better Insights
Better Insights = Better Decisions
Better Decisions = Career Growth!

★ CONSISTENT PRACTICE = STRONG SQL SKILLS ★ | Stay Curious. Keep Exploring Data! – Vinayak Ojha

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

PRACTICE DAILY. THINK DEEPLY. BECOME A SQL EXPERT!

19. STRING FUNCTIONS

FUNCTION	DESCRIPTION	EXAMPLE	RESULT
UPPER()	Converts string to upper case	`SELECT UPPER('vinayak ojha');`	VINAYAK OJHA
LOWER()	Converts string to lower case	`SELECT LOWER('VINAYAK OJHA');`	vinayak ojha
LENGTH()	Returns length of string	`SELECT LENGTH('Vinayak Ojha');`	12
TRIM()	Removes spaces from both sides	`SELECT TRIM('  Vinayak Ojha  ');`	Vinayak Ojha
LTRIM()	Removes spaces from left side	`SELECT LTRIM('  Vinayak Ojha');`	Vinayak Ojha
RTRIM()	Removes spaces from right side	`SELECT RTRIM('Vinayak Ojha  ');`	Vinayak Ojha
SUBSTRING()	Extracts part of string	`SELECT SUBSTRING('Vinayak Ojha', 1, 8);`	Vinayak
REPLACE()	Replaces part of string with another	`SELECT REPLACE('Vinayak Ojha', 'Ojha', 'Kumar');`	Vinayak Kumar
CONCAT()	Joins two or more strings	`SELECT CONCAT('Vinayak', ' ', 'Ojha');`	Vinayak Ojha

20. NUMERIC FUNCTIONS

FUNCTION	DESCRIPTION	EXAMPLE
ROUND()	Rounds to specified decimal places	ROUND(123.456, 2)
CEIL()	Rounds up to nearest integer	CEIL(123.1)
FLOOR()	Rounds down to nearest integer	FLOOR(123.9)
ABS()	Returns absolute value	ABS(-45)
MOD()	Returns remainder of division	MOD(10, 3)
POWER()	Returns power (x raised to y)	POWER(2, 3)

21. CONDITIONAL FUNCTIONS

FUNCTION	DESCRIPTION	EXAMPLE
IF()	Returns value if condition is true	IF(10>5,'Yes','No')
IIF()	Short form of IF()	IIF(10>5,'Yes','No')
CASE	Multiple condition handling	CASE WHEN marks>=60 THEN 'Pass' ELSE 'Fail' END
COALESCE()	Returns first non-NULL value	COALESCE(NULL,'A','B')
NULLIF()	Returns NULL if two values are equal	NULLIF(10,10)

22. NULL FUNCTIONS

FUNCTION	DESCRIPTION	EXAMPLE
IS NULL	Checks if value is NULL	col IS NULL
IS NOT NULL	Checks if value is NOT NULL	col IS NOT NULL
COALESCE()	Returns first non-NULL value	COALESCE(NULL,'X')
IFNULL()	Returns value if not NULL	IFNULL(col, 'N/A')
NULLIF()	Returns NULL if two values equal	NULLIF(col1, col2)

23. SUBQUERY TYPES

SCALAR SUBQUERY
Returns a single value.

```
SELECT name FROM employees
WHERE salary = (SELECT MAX(salary)
FROM employees);
```

MULTI-ROW SUBQUERY
Returns multiple rows.

```
SELECT name FROM employees
WHERE dept_id IN (SELECT id FROM
departments);
```

CORRELATED SUBQUERY
Refers to outer query.

```
SELECT name FROM employees e1
WHERE salary > (SELECT AVG(salary)
FROM employees e2 WHERE e2.dept_id =
e1.dept_id);
```

EXISTS SUBQUERY
Checks existence of rows.

```
SELECT name FROM employees e
WHERE EXISTS (SELECT 1 FROM orders o
WHERE o.emp_id = e.id);
```

24. WINDOW FUNCTIONS (ANALYTIC FUNCTIONS)

ROW_NUMBER()	Assigns unique row number in a result set
RANK()	Ranks rows with gaps
DENSE_RANK()	Ranks rows without gaps
NTILE(n)	Divides rows into n equal groups
LAG()	Accesses data from previous row
LEAD()	Accesses data from next row
SUM() OVER()	Running total
AVG() OVER()	Running average

EXAMPLE:

```
SELECT name, salary,
ROW_NUMBER() OVER
(ORDER BY salary DESC) AS rn,
RANK() OVER (ORDER BY salary
DESC) AS rnk
FROM employees;
```

TIP:
Window functions do not reduce rows. They perform calculations across a set of table rows.

QUICK TIPS

- Use meaningful aliases.
- Understand your data before writing complex queries.
- Break complex queries into smaller parts.
- Practice subqueries and joins regularly.
- Test queries with small datasets first.

COMMON MISTAKES

- Forgetting WHERE clause in JOIN.
- Using SELECT * in production queries.
- Not handling NULL values properly.
- Missing GROUP BY with aggregate functions.
- Not using indexes for large tables.

BEST PRACTICES

- Write readable and well-formatted SQL.
- Use aliases for tables and columns.
- Filter data early (WHERE before GROUP BY).
- Use appropriate data types.
- Comment your SQL for better understanding.

Created by
VINAYAK OJHA
Keep Learning. Keep Growing!

CONNECT WITH ME
linkedin.com/in/vinayak-ojha-845854322
vinayakojha2002@gmail.com

REMEMBER
Good SQL skills = Better Insights
Better Insights = Better Decisions
Better Decisions = Career Growth!

★ CONSISTENT PRACTICE = STRONG SQL SKILLS ★ | Stay Curious. Keep Exploring Data! ♥ – Vinayak Ojha

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

PRACTICE. ANALYZE. BUILD REAL-WORLD DATA SKILLS.

25. AGGREGATE FUNCTIONS (ADVANCED)

Used to perform calculations on a set of values and return a single result.

FUNCTION	DESCRIPTION	EXAMPLE
COUNT()	Counts rows (including NULLs).	`SELECT COUNT(*) FROM employees;`
COUNT(DISTINCT)	Counts unique values.	`SELECT COUNT(DISTINCT dept) FROM employees;`
SUM()	Returns total sum.	`SELECT SUM(salary) FROM employees;`
AVG()	Returns average value.	`SELECT AVG(salary) FROM employees;`
MIN()	Returns minimum value.	`RELECT MIN(salary) FROM employees;`
MAX()	Returns maximum value.	`SELECT MAX(salary) FROM employees;`

26. GROUP BY & HAVING

GROUP BY groups rows with same values.
HAVING filters groups based on a condition.

CLAUSE	PURPOSE
GROUP BY	Groups rows with same values.
HAVING	Filters groups (used with aggregate functions).

EXAMPLE:

```
SELECT dept, COUNT(*) AS total_emp,
AVG(salary) AS avg_salary
FROM employees
GROUP BY dept
HAVING AVG(salary) > 50000;
```

27. SET OPERATIONS

Used to combine results from two or more queries.

UNION

Returns distinct rows from both queries.

EXAMPLE:

```
SELECT name FROM emp1
UNION
SELECT name FROM emp2;
```

UNION ALL

Returns all rows (including duplicates).

EXAMPLE:

```
SELECT name FROM emp1
UNION ALL
SELECT name FROM emp2;
```

INTERSECT

Returns common rows between queries.

EXAMPLE:

```
SELECT name FROM emp1
INTERSECT
SELECT name FROM emp2;
```

EXCEPT (MINUS)

Returns rows from first query that are not in second query.

EXAMPLE:

```
SELECT name FROM emp1
EXCEPT
SELECT name FROM emp2;
```

28. SUBQUERY IN FROM (DERIVED TABLE)

A subquery in the FROM clause creates a temporary table.

EXAMPLE:

```
SELECT dept, avg_salary
FROM (
      SELECT dept, AVG(salary) AS avg_salary
      FROM employees
      GROUP BY dept
) AS dept_avg
WHERE avg_salary > 50000;
```

Subquery (Temporary Table) → Main Query (Outer Query)

29. CASE STATEMENT (ADVANCED)

Used for conditional logic and data transformation.

EXAMPLE:

```
SELECT name, salary,
  CASE
    WHEN salary >= 70000 THEN 'High'
    WHEN salary >= 50000 THEN 'Medium'
    ELSE 'Low'
  END AS salary_level
FROM employees;
```

MORE EXAMPLES:

- Categortize values (e.g., grades, performance levels).
- Apply multiple conditions using `WHEN ... THEN`.
- Nested CASE statements for complex logic.

30. WINDOW FUNCTIONS

Used for running calculations across a set of rows related to the current row.

FUNCTION	DESCRIPTION
ROW_NUMBER()	Assigns a unique number to each row.
RANK()	Ranks rows (with gaps).
DENSE_RANK()	Ranks rows (without gaps).
LAG()	Accesses data from previous row.
LEAD()	Accesses data from next row.
SUM() OVER()	Running total.
AVG() OVER()	Running average.

EXAMPLE:

```
SELECT name, salary,
 ROW_NUMBER() OVER(
   ORDER BY salary DESC
 ) AS rnk
FROM employees;
```

TIP:
Windows functions do not reduce rows. They perform calculations across a set of table rows.

QUICK TIPS

- Use indexes for faster query execution.
- Avoid **SELECT *** in production queries.
- Use **WHERE** for filtering, **HAVING** for groups.
- Use **LIMIT** to restrict rows.
- Keep queries simple and readable.
- Always validate data before analysis.

COMMON MISTAKES

- Forgetting WHERE clause in JOIN.
- Using **SELECT *** in production queries.
- Not using GROUP BY with aggregate functions.
- Using wrong data types.
- Ignoring NULL values.

BEST PRACTICES

- Write optimized and readable queries.
- Use aliases for tables and columns.
- Filter data early (WHERE before GROUP BY).
- Use appropriate data types.
- Test queries with small datasets first.
- Comment complex queries.

Created by
VINAYAK OJHA
Keep Learning. Keep Growing!

Connect with me
linkedin.com/in/vinayak-ojha-845854322
vinayakojha2002@gmail.com

Remember
Good SQL skills = Better Insights
Better Insights = Better Decisions
Better Decisions = Career Growth!

★ CONSISTENT PRACTICE = STRONG SQL SKILLS ★ Stay Curious. Keep Exploring Data! – Vinayak Ojha

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

PRACTICE DAILY. MASTER SQL. BUILD AMAZING SOLUTIONS!

31. INDEXES

Indexes improve query performance by speeding up data retrieval.

TYPES OF INDEXES

PRIMARY INDEX	Automatically created on PRIMARY KEY.
UNIQUE INDEX	Ensures all values in a column are unique.
COMPOSITE INDEX	Index on two or more columns.
CLUSTERED INDEX	Sorts and stores data rows based on index order.
NON-CLUSTERED INDEX	Stores index separately from the actual data.

EXAMPLE:

```
-- Create Index
CREATE INDEX idx_name
ON employees(name);

-- Composite Index
CREATE INDEX idx_dept_salary
ON employees(dept, salary);

-- Drop Index
DROP INDEX idx_name
ON employees;
```

32. CONSTRAINTS

Constraints enforce rules on data in a table.

CONSTRAINT	DESCRIPTION
PRIMARY KEY	Uniquely identifies each record.
FOREIGN KEY	Ensures referential integrity between tables.
UNIQUE	Ensures all values in a column are unique.
NOT NULL	Ensures a column cannot have NULL values.
CHECK	Ensures all values satisfy a specific condition.
DEFAULT	Sets a default value for a column.

EXAMPLE:

```
CREATE TABLE employees (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  dept_id INT,
  salary DECIMAL(10,2) CHECK (salary > 0),
  dept_name VARCHAR(50) DEFAULT 'Unknown',
  FOREIGN KEY (dept_id) REFERENCES departments(id)
);
```

33. TRANSACTIONS

Transactions ensure data consistency and integrity.

ACID PROPERTIES

ATOMICITY	All or nothing.
CONSISTENCY	Data remains valid.
ISOLATION	Transactions are independent.
DURABILITY	Changes are permanent.

EXAMPLE:

```
BEGIN TRANSACTION;
UPDATE accounts  SET balance = balance - 100
WHERE id = 1;
UPDATE accounts  SET balance = balance + 100
WHERE id = 2;
COMMIT; -- Saves all changes
-- ROLLBACK; -- Undoes changes
```

Use transactions when working with critical data to avoid partial updates.

34. DATE & TIME FUNCTIONS

FUNCTION	DESCRIPTION	EXAMPLE
NOW()	Current date and time	SELECT NOW();
CURDATE()	Current date	SELECT CURDATE();
CURTIME()	Current time	SELECT CURTIME();
DATE()	Extract date part	SELECT DATE('2025-05-18 10:30:00');
YEAR()	Extract year	SELECT YEAR('2025-05-18');
MONTH()	Extract month (1-12)	SELECT MONTH('2025-05-18');
DAY()	Extract day (1-31)	SELECT DAY('2025-05-18');
DATEDIFF(d1, d2)	Days between d1 and d2	SELECT DATEDIFF('2025-05-20', '2025-05-18');

35. ORDER BY, LIMIT & OFFSET

Used to sort and limit result sets.

ORDER BY

Sorts the result set.

EXAMPLE:

```
SELECT * FROM employees
ORDER BY salary DESC;
```

LIMIT

Limits the number of rows.

EXAMPLE:

```
SELECT * FROM employees
LIMIT 10;
```

OFFSET

Skips a number of rows.

EXAMPLE:

```
SELECT * FROM employees
LIMIT 10 OFFSET 20;
```

Use ORDER BY with LIMIT and OFFSET for pagination.

36. DATA TYPES (COMMON)

DATA TYPE	DESCRIPTION	EXAMPLE
INT	Whole number	123
BIGINT	Large integer	1234567890123
VARCHAR(n)	Variable length string	'Vinayak Ojha'
CHAR(n)	Fixed length string	'OJHA'
DATE	Date (YYYY-MM-DD)	'2025-05-18'
DATETIME	Date and time	'2025-05-18 10:30:00'
DECIMAL(p,s)	Exact numeric	12345.67
BOOLEAN	True / False	TRUE
TEXT	Large text	'This is a long text'

37. WILDCARDS

WILDCARD	DESCRIPTION
%	Represents zero or more characters
_	Represents a single character
[abc]	Any one character between a, b or c
[^abc]	Any character except a, b or c

EXAMPLE:

```
SELECT * FROM employees
WHERE name LIKE 'A%';
-- Names starting with A

SELECT * FROM employees
WHERE name LIKE '_i%';
-- Second letter is i

SELECT * FROM employees
WHERE name LIKE '[A-C]%';
-- Names starting with A, B or C
```

38. USEFUL TIPS

- Always use meaningful alias names.
- Use WHERE before GROUP BY.
- Nse HAVING to filter groups.
- Normalize your database.
- Avoid SELECT * in large tables.
- Use indexes for faster queries.
- Test queries with small data first.
- Comment your SQL for better understanding.

QUICK RECAP

- **SELECT** – Retrieve data
- **WHERE** – Filter data
- **GROUP BY** – Group rows
- **HAVING** – Filter groups
- **ORDER BY** – Sort result
- **JOIN** – Combine tables
- **INSERT** – Add data
- **UPDATE** – Modify data
- **DELETE** – Remove data
- **DISTINCT** – Unique values
- **COUNT, SUM, AVG** – Aggregate data
- **LIKE** – Search with patterns

BEST PRACTICES

- Write clean, readable and well-formatted SQL.
- Use meaningful names for tables and columns.
- Use appropriate data types.
- Index important columns.
- Regularly backup your database.
- Optimize queries for performance.
- Keep security in mind (avoid SQL Injection).

COMMON MISTAKES

- Forgetting WHERE clause in JOIN.
- Using **SELECT** * in production.
- Not handling NULL values.
- Missing GROUP BY with aggregates.
- Not using indexes.
- Using subqueries unnecessarily.
- Ignoring error messages.

Created by
VINAYAK OJHA
Keep Learning. Keep Growing!
linkedin.com/in/vinayak-ojha-845854322
vinayakojha2002@gmail.com

Good SQL skills =
Better Insights
Better Decisions
Better Career!

SQL CHEAT SHEET

QUERIES • FUNCTIONS • JOINS • ANALYTICS

PRACTICE DAILY. MASTER SQL. BECOME A DATA EXPERT!

39. SAMPLE PRACTICE QUERIES

TASK	QUERY
1. All columns from employees	`SELECT * FROM employees;`
2. Employees in 'Sales' department	`SELECT * FROM employees WHERE department = 'Sales';`
3. Top 5 highest salaries	`SELECT * FROM employees ORDER BY salary DESC LIMIT 5;`
4. Total salary by department	`SELECT department, SUM(salary) AS total_salary FROM employees GROUP BY department;`
5. Employees hired after 2023-01-01	`SELECT * FROM employees WHERE hire_date > '2023-01-01';`
6. Duplicate departments	`SELECT department, COUNT(*) AS cnt FROM employees GROUP BY department HAVING COUNT(*) > 1;`
7. Departments with avg salary > 50000	`SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY department HAVING AVG(salary) > 50000;`

40. COMMON SQL OPERATORS

TYPE	OPERATORS
Arithmetic	`+ - * / %`
Comparison	`= != <> > < >= <=`
Logical	`AND OR NOT`
Bitwise	`& | ^ ~ << >>`
Membership	`IN NOT IN`
Pattern Matching	`LIKE NOT LIKE`
Range	`BETWEEN AND`
Null Check	`IS NULL IS NOT NULL`

OPERATOR EXAMPLE:

```
SELECT * FROM employees
WHERE salary >= 50000 AND department IN ('Sales', 'HR')
AND name LIKE 'A%';
```

41. NORMALIZATION (BASICS)

Normalization reduces data redundancy and improves integrity.

NORMAL FORM	DESCRIPTION
1NF (First NF)	Eliminate repeating groups. Each column should have atomic values.
2NF (Second NF)	Be in 1NF and remove partial dependency. Non-key attributes must depend on the whole key.
3NF (Third NF)	Be in 2NF and remove transitive dependency. Non-key attributes should depend only on the key.
BCNF	Stronger than 3NF. Every determinant must be a candidate key.

42. SQL JOINS SUMMARY

JOIN TYPE	DESCRIPTION	SYNTAX EXAMPLE
INNER JOIN	Returns matching rows from both tables	`SELECT * FROM A INNER JOIN B ON A.id = B.id;`
LEFT JOIN	All rows from left table + matching from right	`SELECT * FROM A LEFT JOIN B ON A.id = B.id;`
RIGHT JOIN	All rows from right table + matching from left	`SELECT * FROM A RIGHT JOIN B ON A.id = B.id;`
FULL OUTER JOIN	All rows from both tables	`SELECT * FROM A FULL OUTER JOIN B ON A.id = B.id;`
CROSS JOIN	Cartesian product (all combinations)	`SELECT * FROM A CROSS JOIN B;`

43. DATA ANALYTICS SQL PATTERNS

PATTERN	USE CASE
Running Total	`SUM(value) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`
Moving Average	`AVG(value) OVER (ORDER BY date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
Rank	`RANK() OVER (PARTITION BY dept ORDER BY salary DESC)`
Percent Rank	`PERCENT_RANK() OVER (ORDER BY salary)`
Lead / Lag	`LEAD(value) OVER (ORDER BY date)` `LAG(value) OVER (ORDER BY date)`

44. SQL BEST PRACTICES

- Use meaningful and consistent alias names.
- Write clean, readable and well-indented SQL.
- Use WHERE before GROUP BY.
- Filter data early to improve performance.
- Avoid SELECT * in production queries.
- Use appropriate indexes for large tables.
- Test queries with small datasets first.
- Comment your SQL for better understanding.
- Regularly backup your database.

QUICK REMINDERS

- SELECT – Retrieve data
- WHERE – Filter data
- GROUP BY – Group rows
- HAVING – Filter groups
- ORDER BY – Sort result
- JOIN – Combine tables
- AGGREGATE – Summarize data
- SUBQUERY – Nested query
- INDEX – Speed up queries

COMMON MISTAKES

- Forgetting WHERE clause in JOIN.
- Using SELECT * in production.
- Not handling NULL values.
- Missing GROUP BY with aggregate functions.
- Not using indexes for large tables.
- Ignoring error messages.

SQL IS POWERFUL!

The more you practice, the better you become.

Practice → Analyze → Build → Succeed!

Created by
VINAYAK OJHA
Keep Learning. Keep Growing!

CONNECT WITH ME
linkedin.com/in/vinayak-ojha-845854322
vinayakojha2002@gmail.com

REMEMBER

Good SQL skills = Better Insights
Better Insights = Better Decisions
Better Decisions = Career Growth!